

ARX MDM

ARX MDM is a unified management platform: a **Go** HTTP/WebSocket API server, a **Vite/React** operator dashboard, **PostgreSQL** persistence, and an **embedded PKI** (root + intermediate CA) built into the server for mutual TLS with enrolled agents.

On **first startup**, if the configured PKI storage directory has no CA material, the server generates an **ECDSA P-384** root CA and intermediate CA, writes PEM keys with restrictive permissions, and refreshes `mtls-client-ca-bundle.pem` (intermediate + root) for use as `ARX_MTLS_CLIENT_CA_BUNDLE`. Enrollment (`POST /v1/enroll`) signs agent CSRs with the intermediate CA and returns the **client certificate chain** plus the **root CA** PEM.

Area	Details
Server	<code>cmd/server</code> — REST, WebSocket C2, JWT dashboard auth, embedded CA in <code>internal/pki</code>
Agent	<code>cmd/agent</code> — telemetry, enrollment, optional mTLS (<code>internal/agent</code>)
Dashboard	<code>web/</code> — Vite + React
Persistence	PostgreSQL 15+; versioned SQL migrations embedded in the server (<code>internal/database/migrations/</code> , applied at startup via <code>internal/database/migrations.go</code>)
PKI / mTLS	Embedded CA (<code>internal/pki/ca.go</code>); storage path <code>ARX_PKI_STORAGE_PATH</code> (default <code>certs</code> under the process working directory)
Unified image	Multi-stage <code>Dockerfile</code> : Vite build is copied into <code>internal/serverinstall/dashboard</code> and embedded in <code>arx-server</code> with <code>-tags embedbins</code> ; Linux and Windows agents are embedded and served from <code>/v1/install/bin/*</code>

Production server (recommended)

The default production path is **two containers** (`postgres` + `arx-server`) from `docker-compose.yml`. The Postgres image only provisions an empty database; **arx-server** **applies all schema migrations automatically** on startup (and before any `admin` CLI subcommand that uses the database). Reusing an existing `postgres_data` volume is supported: the migration runner records applied versions in `arx_schema_migrations` and only runs pending steps.

One command on Ubuntu (bootstrap)

From the repository root on the host (as **root** so Docker and privileged steps work):

```
chmod +x scripts/bootstrap_server.sh
sudo ./scripts/bootstrap_server.sh
```

The script installs `docker.io` and `docker-compose-plugin` if they are missing, writes a **mode-600** `.env` with generated `POSTGRES_PASSWORD`, `ARX_JWT_SECRET`, and `ARX_BOOTSTRAP_ADMIN_PASSWORD`, sets `ARX_DASHBOARD_ORIGINS` from the host's primary IPv4 address and localhost, then runs `docker compose up -d --build`.

- **Dashboard and API**: `http://<host>:8080/` (same origin; API under `/v1/...`).
- **Bootstrap admin**: the `compose .env` includes `ARX_BOOTSTRAP_ADMIN_PASSWORD` so the first HTTP server start can create the initial admin via `BootstrapAdminIfEmpty`, or you can use **admin setup** instead (see [Server CLI](#)).

Optional: set `ARX_PUBLISH_PORT` before the script (default `8080`) to change the host port mapping.

Server CLI (inside the `arx-server` container)

The binary uses **Cobra** subcommands. The default with no arguments is the same as **serve**: start the HTTP/WebSocket server. Database admin commands require `ARX_DATABASE_URL` (and run migrations first). PKI commands only need `ARX_PKI_STORAGE_PATH` (default `certs`).

```
docker compose exec arx-server /app/arx-server help
docker compose exec arx-server /app/arx-server serve
docker compose exec arx-server /app/arx-server admin setup
docker compose exec arx-server /app/arx-server admin create-user
docker compose exec arx-server /app/arx-server admin create-token
docker compose exec arx-server /app/arx-server admin reset-password someuser
docker compose exec arx-server /app/arx-server pki bootstrap
```

- **serve** (or no subcommand): Runs the MDM API after applying pending DB migrations.
- **admin setup**: If the `users` table already has at least one **admin** row, the command exits successfully after printing a short message to `stderr`. Otherwise it creates user **admin** with a random password and prints `username / password` to `stdout`.
- **admin create-user**: Prompts on `stderr` for username, role (`admin`, `operator`, `viewer`), and password (hidden when `stdin` is a TTY), then inserts the row.

- **admin create-token** : Inserts a row into `enrollment_tokens` (SHA-256 hash of a new 16-byte random secret, 7-day expiry). The **presentation secret** alone is printed to stdout (one line) for use as `ARX_ENROLL_TOKEN` in install scripts; token metadata is printed to stderr.
- **admin reset-password USERNAME** : Prompts for a new password twice and updates the bcrypt hash (username match is case-insensitive).
- **pki bootstrap** : Ensures root and intermediate CA PEMs exist under `ARX_PKI_STORAGE_PATH` (same logic as server startup).

To use **admin setup** as the only first-admin path, remove `ARX_BOOTSTRAP_ADMIN_USERNAME` and `ARX_BOOTSTRAP_ADMIN_PASSWORD` from `.env` before the first server start (the bootstrap script adds them by default, so edit `.env` after bootstrap or adjust the script).

Makefile (developer / CI)

Target	Purpose
<code>make sync-install-scripts</code>	Copy <code>scripts/install_*.{sh,ps1}</code> into <code>internal/serverinstall/</code> for go:embed.
<code>make build-all</code>	Sync scripts, <code>npm ci && npm run build</code> in <code>web/</code> , copy <code>web/dist</code> into <code>internal/serverinstall/dashboard</code> , cross-build embedded agents, build <code>bin/arx-server</code> with <code>-tags embedbins</code> .
<code>make docker-up</code>	<code>docker compose up -d --build</code> .
<code>make clean</code>	Remove <code>bin/</code> , <code>web/dist</code> , and generated embed inputs under <code>internal/serverinstall/</code> .

Client zero-touch install

Create an **enrollment token** from the running server container (captures the secret into a shell variable):

```
SECRET="$(docker compose exec -T arx-server /app/arx-server admin create-token)"
```

Linux (systemd)

```
curl -fsSL "http://<mdm-host>:8080/v1/install/linux" | sudo \
  env ARX_SERVER_URL="http://<mdm-host>:8080" ARX_ENROLL_TOKEN="$SECRET" bash
```

Use `https://` and omit `ARX_INSECURE_TLS` when the server presents a proper TLS certificate. For lab HTTP or self-signed HTTPS downloads only, set `ARX_INSECURE_TLS=1`.

Windows (elevated PowerShell)

```
$env:ARX_SERVER_URL = 'http://<mdm-host>:8080'
$env:ARX_ENROLL_TOKEN = (docker compose exec -T arx-server /app/arx-server admin create-token).Trim()
irm 'http://<mdm-host>:8080/v1/install/windows' | iex
```

The scripts download the agent from `/v1/install/bin/linux` or `/v1/install/bin/windows`, enroll with `enroll -server ... -token ...`, install the service (systemd on Linux, native SCM on Windows), and start the agent.

Public install endpoints

Method	Path	Purpose
GET	<code>/v1/install/linux</code>	Shell installer (no secrets embedded; uses environment variables).
GET	<code>/v1/install/windows</code>	PowerShell installer.
GET	<code>/v1/install/bin/linux</code>	Embedded <code>arx-agent</code> (Linux amd64).
GET	<code>/v1/install/bin/windows</code>	Embedded <code>arx-agent.exe</code> (Windows amd64).

TLS and telemetry (production hardening)

Compose starts the server on **plain HTTP** by default so you can bring the stack up without certificates. For mutual TLS on telemetry and HTTPS for the dashboard, mount PEM files under `/data/pki` (or another volume) and set `ARX_TLS_CERT`, `ARX_TLS_KEY`, and `ARX_MTLS_CLIENT_CA_BUNDLE` on `arx-server` after the embedded CA has created material under `ARX_PKI_STORAGE_PATH` (see existing PKI documentation in this file). Until those three variables are set, the server logs a warning and telemetry does not require client certificates.

Local development (optional)

Tool	Notes
Go	Version in <code>go.mod</code> .
Node.js	20+ for <code>web/</code> .
PostgreSQL	15+; point <code>ARX_DATABASE_URL</code> at an empty or existing database—the server migrates schema on startup.

Run the Vite dev server on port `5173` with `ARX_DASHBOARD_ORIGINS` including `http://localhost:5173`. A plain `go build ./cmd/server` embeds a **stub** dashboard page and does **not** embed agent binaries; use `make build-all` or the Docker image for full assets and install artifacts.

With `ARX_DATABASE_URL` exported, `bin/arx-server admin ...` subcommands run without starting the HTTP server (no `ARX_JWT_SECRET` required for `admin`). Use `arx-server pki bootstrap` without a database when you only need CA material on disk.

```
go build -o bin/arx-server ./cmd/server
go build -o bin/arx-agent ./cmd/agent
cd web && npm ci && npm run dev
```

Testing

Go unit tests

```
go test ./...
```

PKI / enrollment smoke (manual)

1. Start Postgres and `arx-server` (migrations apply automatically), or point `ARX_DATABASE_URL` at any reachable Postgres and run `arx-server admin create-token` once.
2. Mint an enrollment token with `docker compose exec -T arx-server /app/arx-server admin create-token` (see [Client zero-touch install](#)).
3. Start `arx-server` with valid `ARX_DATABASE_URL` and writable `ARX_PKI_STORAGE_PATH`.
4. Run an install script or `arx-agent enroll -server ... -token ...`. Expect HTTP `201` from `POST /v1/enroll` and PEM material under the agent cert directory.

10. Backup and restore

`scripts/backup.sh` archives PostgreSQL (`pg_dump -Fc`) and, when available, the embedded PKI directory as `pki/` inside the tarball (host path from `ARX_PKI_STORAGE_PATH`, or `./certs` under the compose directory when populated).

`scripts/restore.sh` restores the database from `postgres.dump` and, when the tarball contains `pki/`, copies it to `ARX_PKI_STORAGE_PATH`. Legacy tarballs that still contain `step-ca/` are also recognized.

11. Security operations notes

- **Protect `ARX_PKI_STORAGE_PATH`**: anyone with the intermediate private key can mint client certificates. Restrict filesystem ACLs and backups.
- **Root distribution**: agents receive `root_ca` in the enroll response to anchor TLS trust; operators must distribute trust deliberately—same as any private PKI.

Repository map (short)

Path	Role
cmd/server	Cobra CLI (<code>serve</code> , <code>admin</code> , <code>pki</code>) and HTTP API entryptoint
cmd/agent	Managed endpoint agent
internal/pki	Embedded root + intermediate CA and CSR signing
internal/api	REST handlers (including <code>enroll.go</code> for <code>POST /v1/enroll</code>)
internal/auth	JWT, enrollment coordinator, CSR validation
internal/cli	Interactive and scripted admin helpers used by <code>arx-server admin ...</code>
internal/database	Embedded SQL migrations (<code>migrations.go</code> , <code>migrations/*.sql</code>)
internal/models	Go structs mirroring persisted tables
internal/serverinstall	Embedded dashboard + agent artifacts; <code>/v1/install/*</code> routes
web/	Operator dashboard (Vite source; production assets embedded in <code>arx-server</code>)